ASSIGNMENT TWO

QUESTION ONE:

(a) Pick an app you use every day (a banking app, WhatsApp, Jumia, etc.). Describe, step by step, what happens behind the scenes from the moment you tap "Login" until you see your home screen. Mention the frontend, the backend and the database.

When I hit the login button on my jumia app, The email and password that was entered is sent to the backend. Here, the API receives it, scrutinize, package and then check the database for exact match. When a match is found in the database, meaning that I entered right email and password, my data is delivered via the same path and I am granted access into my jumia account where I have access to registered users only content.

(b) Come up with your own analogy for the backend that is NOT a car engine, and explain where your analogy breaks down.

Restaurant kitchen

The backend is the kitchen, where the chefs take your order, process it, combine raw ingredients, and cook the meal. The database is the fridge where all the raw ingredients are organized and stored for the chefs to fetch.

The analogy breaks when the kitchen runs out on ingredients.

QUESTION TWO:

(a) Explain why this statement is wrong. What exactly is a runtime, and what role does Google's V8 engine play inside Node.js?

Node.js is not a programming language. It is just a runtime environment that allows for running of javascript codes outside of the browser. Google's V8 engine is responsible for the compilation of javascript codes directly into machine understandable codes.

(b) JavaScript existed for years before 2009. What problem did Ryan Dahl solve by taking it out of the browser, and what new things could JavaScript do after that?

Well, before 2009, javascript only belong in the browser alone and javascript codes written are only run in there. Ryan Dahl solves this problem by coming up with Node.js and hence averting the dependency on browsers alone to run javascript codes.

QUESTION THREE:

(a) Using a real-life scenario of your own (a restaurant, a bank, a mechanic workshop, etc.), explain how one worker can handle many customers efficiently. Map each part of your scenario to the Event Queue, Event Loop, Thread Pool and Callback.

Using a mechanic workshop scenario, there are personnels specialization. Some people fix gear, some engine etc. The person that attends to the customer/client is the worker here. He meets the client, get their complains and Queue these complains up (event queue) for pickup by the technical personnels (event loop). If the complains is simple, he attends to it immediately for clients and they leave. But if not, they are passed to a special team of engineers (Thread pool), phone number is probably collected (for callback) and they are eventually called to pick it up when done.

(b) Describe a situation in your scenario where this approach would fail badly. What would that look like in a Node.js server?

When a one particular engineer has multiple of his task, the process slows down and eventually, the server crashes.

QUESTION FOUR:

(a) A student writes document.getElementById("name") in a Node.js file and gets an error. Explain why, and name the global object that exists in Node.js instead of window.

The reason for the error is that the student tried to access the global object “document” in javascript runtime environment “node.js” which is only available on the browser. The global object that exists in Niode.js is “global” not “document”.

QUESTION FIVE:

A signup form validates the email and password in the browser using client-side JavaScript. The developer says, "So the backend doesn't need to check again."

(a) Explain why this is a dangerous assumption. Describe how someone could send bad data to the server without ever using the form.

This is very dangerous because the authentication and authorization logics are handled by backend and if this fails, danger looms. Malicious users can try SQL injections or can even try accessing the backend via processes that doesn’t require the form in frontend like postman.

QUESTION SIX: (ANSWER IS IN GITHUB)

(a) Look at this code. Identify every problem, then rewrite it following the naming best practices in the slides.

```
let age = 25;

let myName = "Tunde";

const number = 10;
```

```
let secondPlace = "Ife";

let pi = 3.14;

pi = 3.14159;

let userAge = 30;

userAge = 31;
```

(b) **Explain, in your own words, when you would choose const and when you would choose let. Why do many developers start with const and only change to let when forced to? Also, explain why I asked us to avoid using var.**

const is used when we are sure we don't want the variable to be re-declared nor re-assigned. That is, when we don't what the variable to change. While we use let when the value of the variable could change from time to time.

Var on the other side is hoisted. That is, it is globally scoped. Its is re-declarable and re-assignable too. For these, it can cause hidden unexpected bugs like un-intended re-declaration of already declared variable hence overriding them.

QUESTION SEVEN: (ANSWER IS IN GITHUB)

Run the code below and study the result.

```
const student = { name: "Chidi", score: 50 };
student.score = 90;
console.log(student);

const grades = [1, 2, 3];
grades.push(4);
console.log(grades);

student = { name: "Bola", score: 70 };
```

(a) Which lines work and which one throws an error? If const means "constant", why are the first two allowed? Explain what const actually protects.

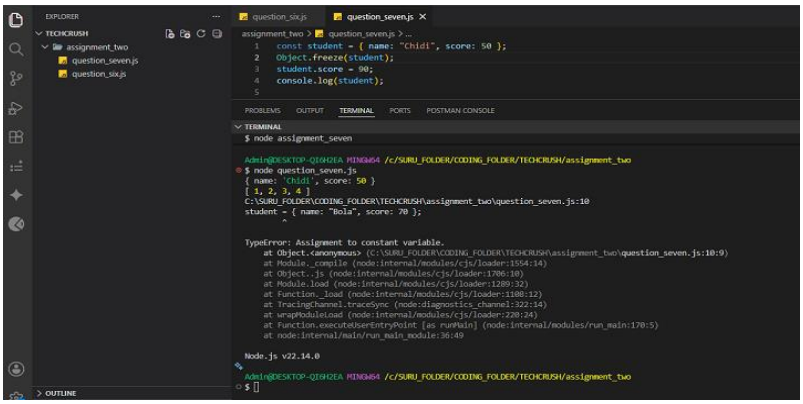
All lines run except line 9 which is the last line.

The first two lines run because the value is an object and not a primitive value. The items in the object can be updated without throwing any error because it is just a reference to where the actual value is stored in memory.

Const protects variables from being assigned a different value completely.

(b) Research Object.freeze() and use it to make the student object truly unchangeable. Show what happens when you try to change a property after freezing, and explain why it fails silently in normal mode.

Object.freeze() protects the values inside an object by making the object itself completely immutable. While const prevents us from changing where the variable points, Object.freeze() prevents us from changing what the object contains.



QUESTION EIGHT: (ANSWER IS IN GITHUB)

The slides say arrays and objects are non-primitives. Predict the output of each block before running it.

```
let a = 10;

let b = a;

b = 20;

console.log(a, b); // 10, 20


let userOne = { name: "Amaka", age: 22 };

let userTwo = userOne;

userTwo.age = 40;

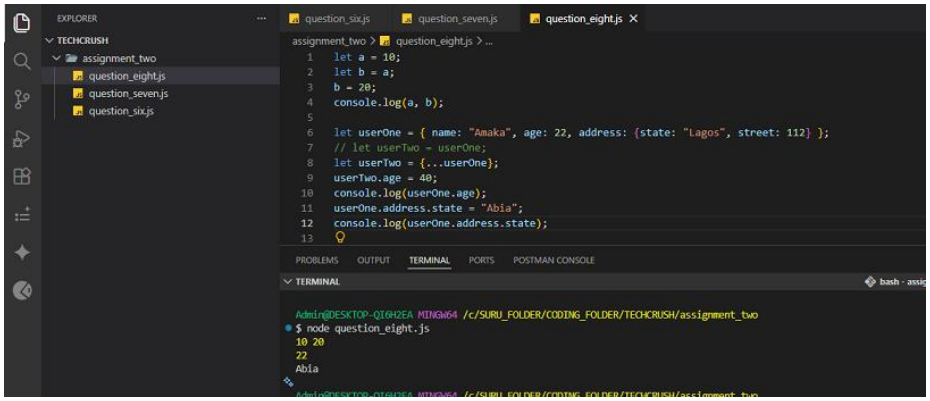
console.log(userOne.age);
```

(a) Explain why the two blocks behave differently. Use the idea of "copying a value" versus "copying a reference".

In objects, the reference is copied not the value unlike in primitive where values are copied.

(b) Fix the second block so that changing userTwo does NOT affect userOne, using the spread operator. Does your fix still work if the object contains another object inside it (for example an address property)? Test it and explain what you found.

The fix work but didn't work with nested objects and the reason is probably because spread makes a shallow copy of the object hence failing to copy the inner object values.



QUESTION NINE: (ANSWER IS IN GITHUB)

(a) Predict the output of typeof for each value, then run it:

42,
"42",
true,
null,
undefined,
[1, 2, 3],
{ a: 1 },
function () {},
NaN.

(b) Two of the results are surprising. Identify them and explain why a developer could get hurt by them in a real backend (for example when checking whether a request body exists).

Typeof null => object
Typeof [1, 2, 3] => object

Request body is an object and the trap of "if (request.body === object)" which will be true for even array and null.

(c) Write a function getType(value) that returns accurate type names such as "null", "array", "object", "number", "string", "function" and "undefined". Test it on all of the values above.

QUESTION TEN (ANSWER IS IN GITHUB)